UNIVERSITY AT ALBANY
STATE UNIVERSITY OF NEW YORK

CSI 436/536 (Fall 2026)

Machine Learning

Lecture 16: Neural Network and Deep Learning

Chong Liu

Department of Computer Science

Nov 10, 2026

Recap: Ensemble classifiers

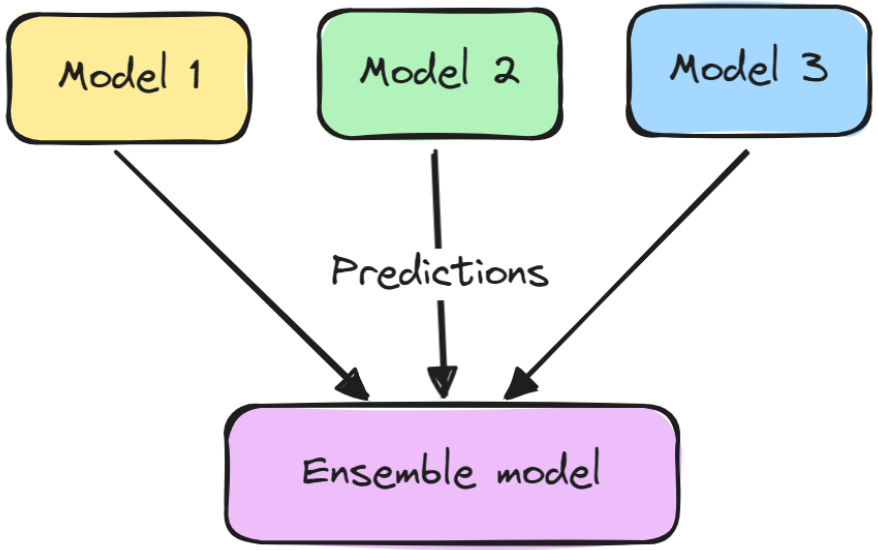
- Take multiple learner $h_1, h_2, h_3, \ldots, h_N$

- Ensemble classifier:
  - For classification problems (output space is discrete)

$$\arg\max_{y \in \mathcal{Y}} \frac{1}{N} \sum_{i=1}^{N} \alpha_i \mathbf{1}(h_i(x) = y)$$

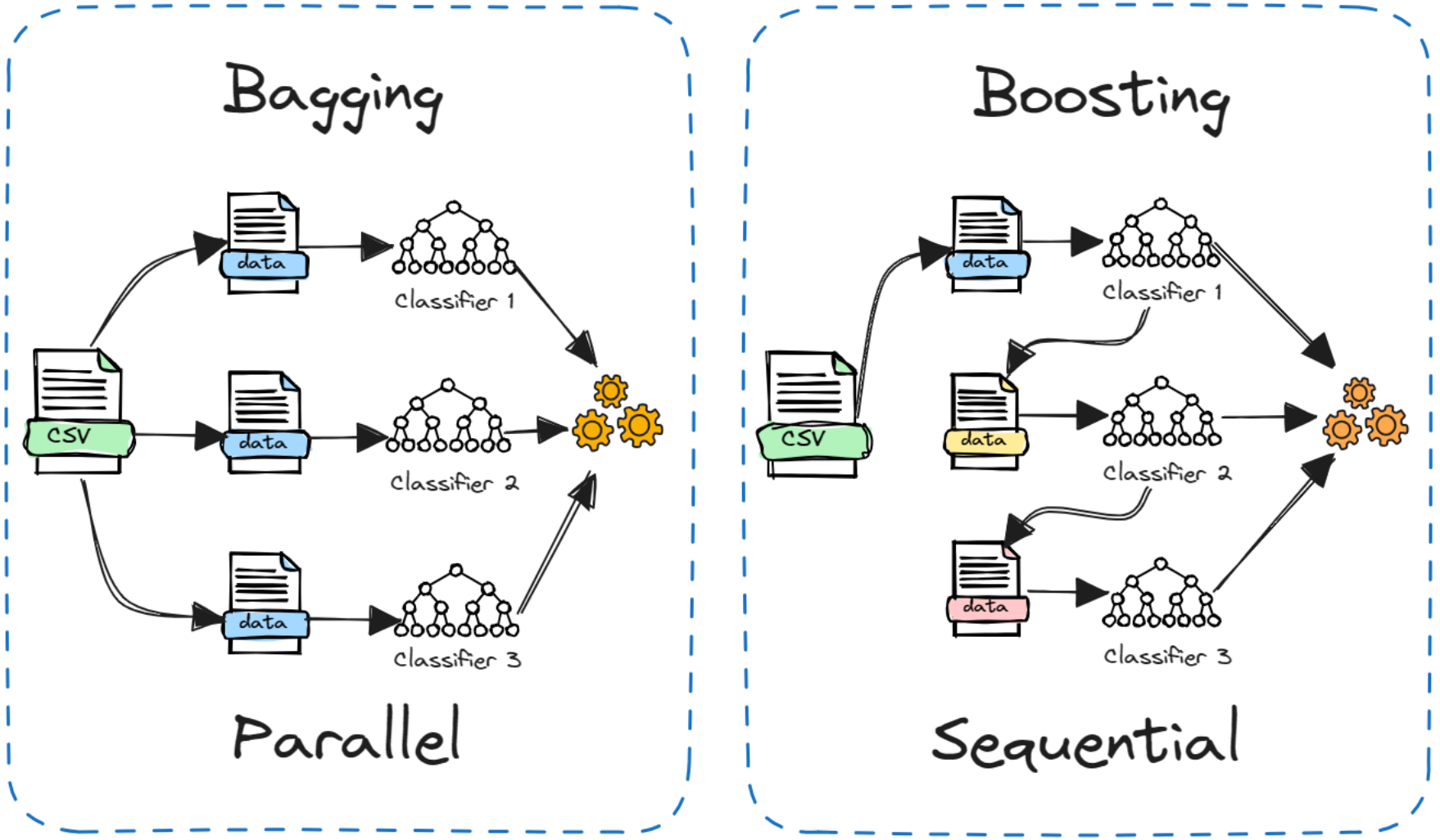
  - For regression problems (output space is continuous)

$$\frac{1}{N} \sum_{i=1}^{N} \alpha_i h_i(x)$$



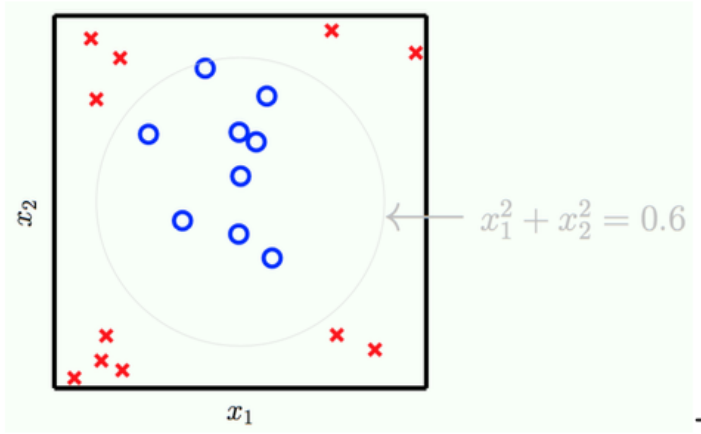
https://www.datacamp.com/tutorial/what-bagging-in-machine-learning-a-guide-with-examples

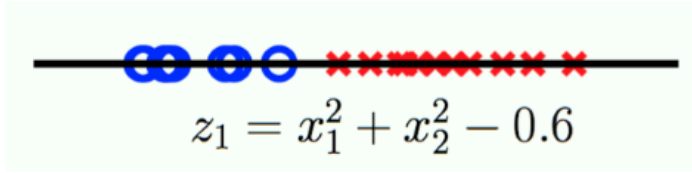
Recap: Bagging and boosting



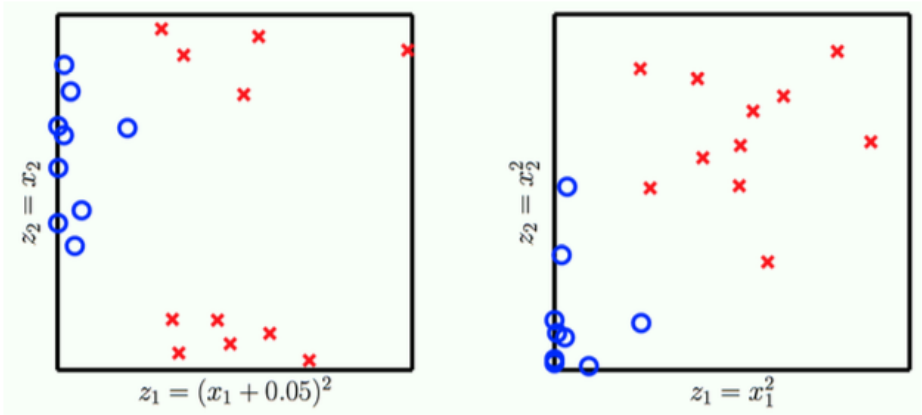
https://www.datacamp.com/tutorial/what-bagging-in-machine-learning-a-guide-with-examples

Recap: Many ways to transform features

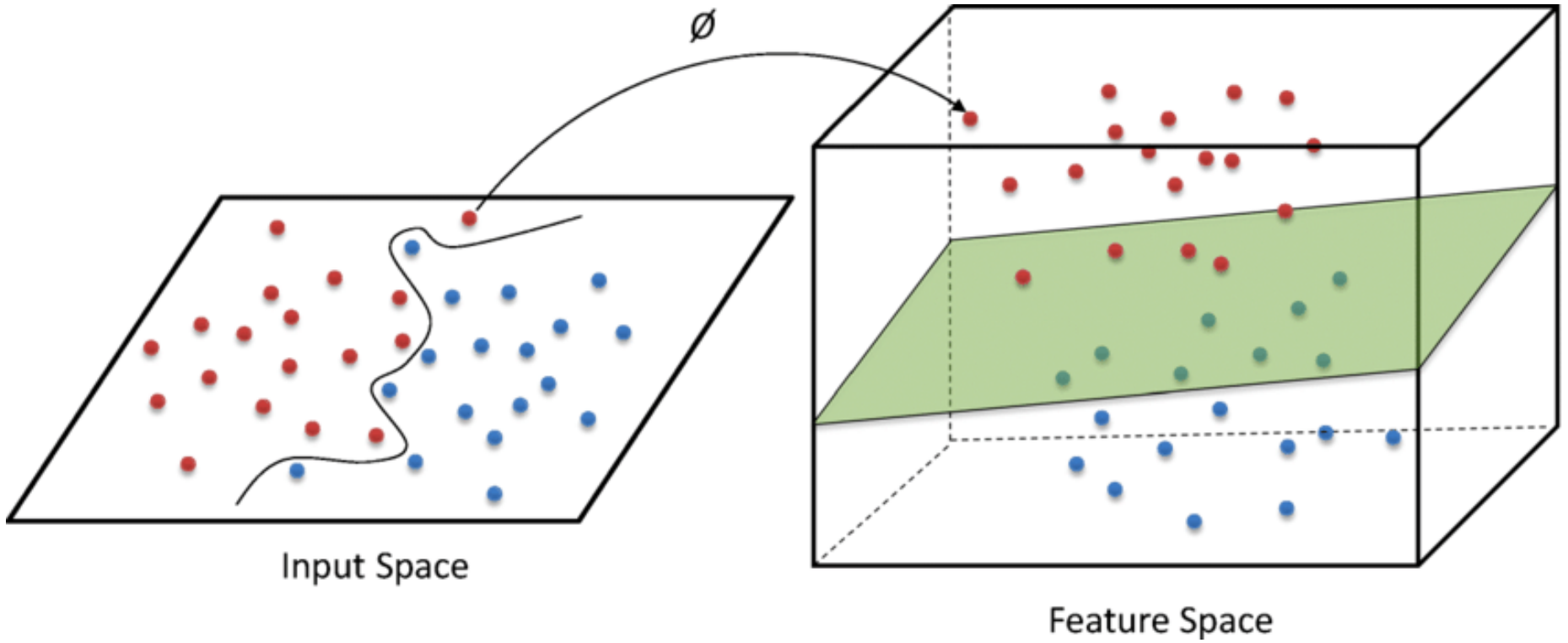




And many other would work ...



Recap: It is easier to linearly classify the data in higher dimensions



Recap: Three main approaches for expanding the hypothesis class (systematically minimizing the approx. error)

- Boosting and Bagging (Ensemble learning)
  - Combine many weak learners (e.g., decision trees with depth 3) into a strong learner

- Kernel methods (lift features to higher-dimensional space)
  - e.g., adding polynomial expansion, add interaction terms
  - Other nonlinear transformation of the original features

- Deep Learning
  - Train large neural networks using SGD
  - Learn feature representation and classification jointly.

Today

- Understand neural network
  - From linear model
  - From kernel method
- From neural network to deep learning
- A brief history of machine learning (starting 1990s)

Example of neural network: AlexNet (2012) – starting point of deep learning

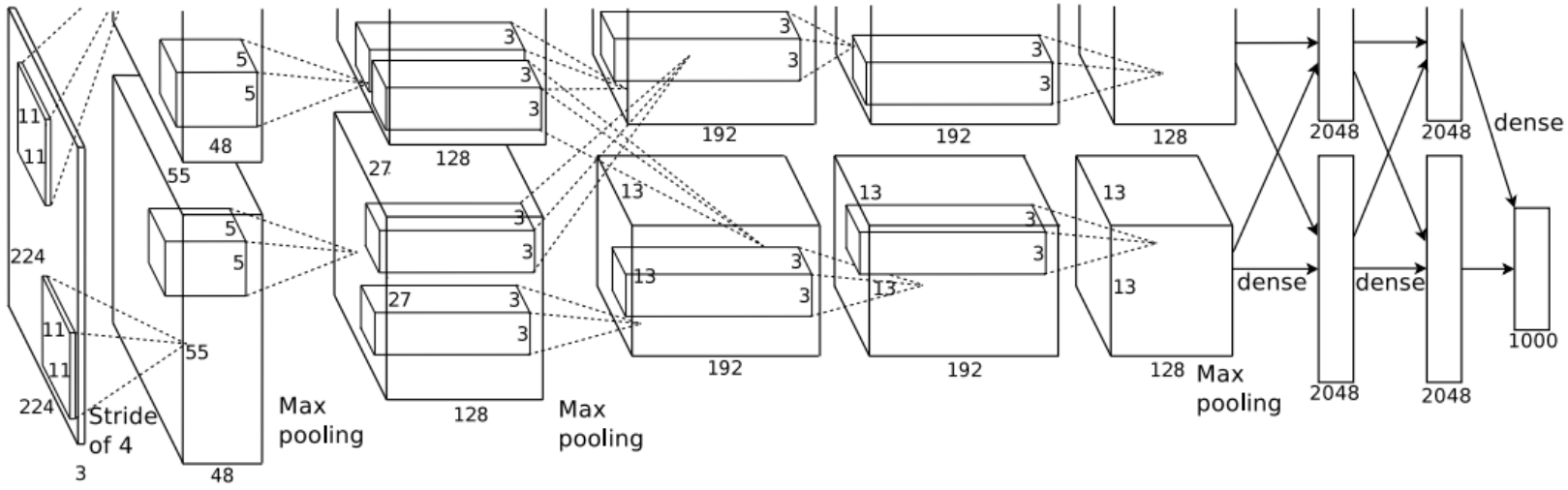


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

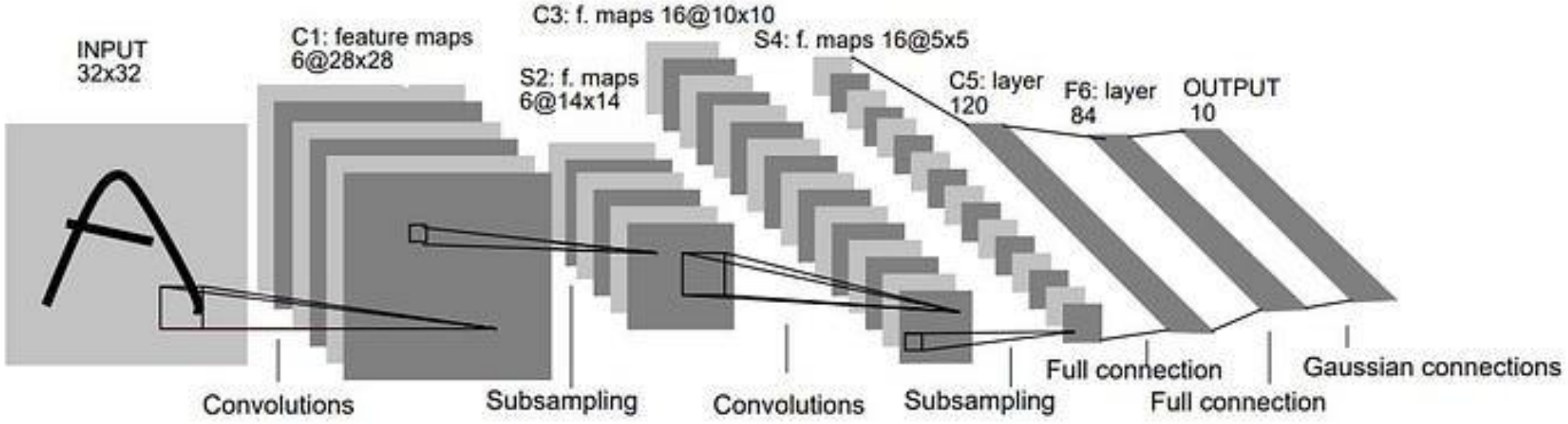
Imagenet classification with **deep convolutional neural** networks

A Krizhevsky, I Sutskever… - Advances in **neural** …, 2012 - proceedings.neurips.cc

… We trained a large, **deep convolutional neural** network to **classify** the 1.2 million high-resolution images in the **ImageNet** LSVRC-2010 contest into the 1000 different classes. On the test …

☆ Save 99 Cite Cited by 160669 Related articles All 81 versions »

LeNet (1998)



Gradient-based learning applied to document recognition

Y LeCun, L Bottou, Y Bengio, P Haffner

Proceedings of the IEEE, 1998 • ieeexplore.ieee.org

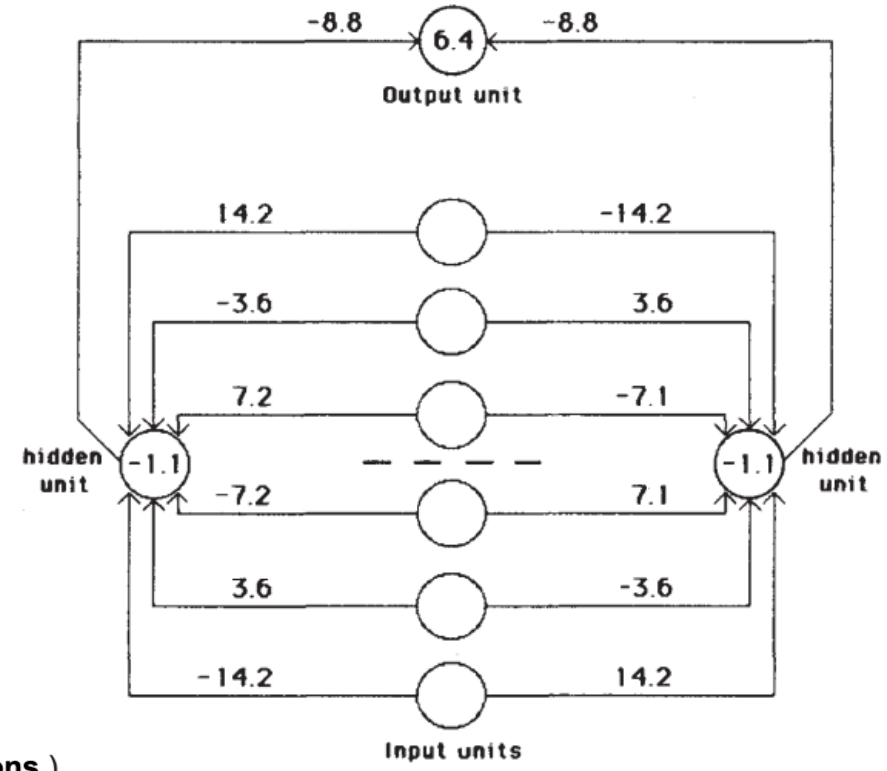
> Multilayer neural networks trained with the back-propagation algorithm constitute the best example of a successful gradient based learning technique. Given an appropriate network architecture, gradient-based learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns, such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit

SHOW MORE

☆ Save 99 Cite Cited by 91977 Related articles All 34 versions

Rumelhart, Hinton, Williams (1986)

- One layer of a feedforward neural networks



Learning representations by **back-propagating errors**

DE Rumelhart, GE Hinton, RJ Williams - nature, 1986 - nature.com

… their states are completely determined by the input vector: they do not **learn representations**.)

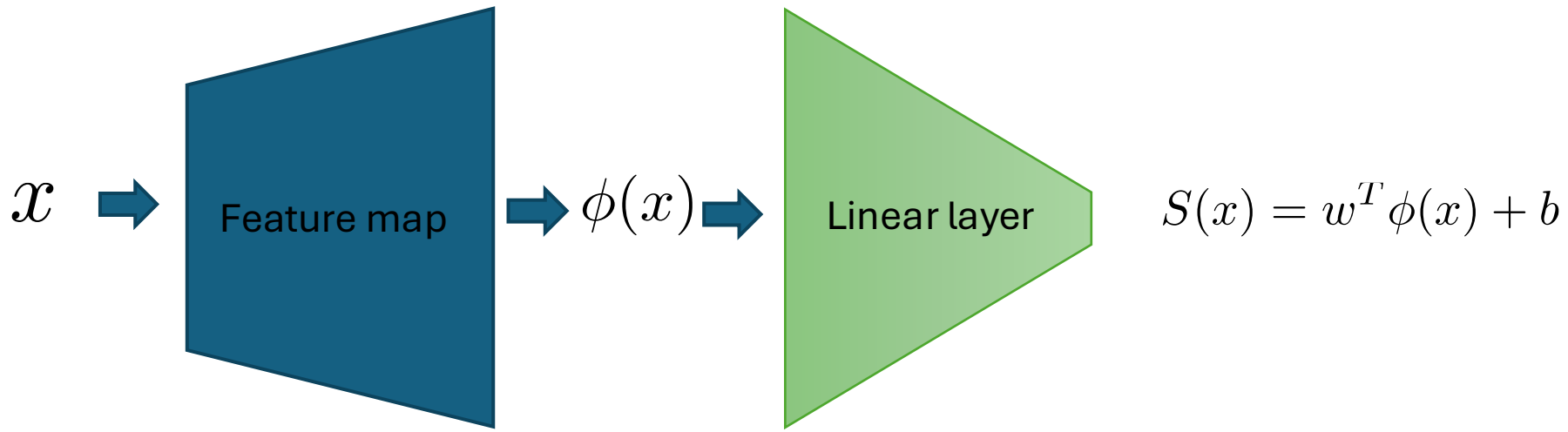
The **learning** procedure must decide under what circumstances the hidden units should be …

☆ Save 99 Cite Cited by 53131 Related articles All 27 versions

It goes back even further…

- 1943 Pitts and McCulloch: Perceptron model to mimic the brain
- 1956: Rosenblatt's Perceptron Implementation
- 1960s:
  - Ivakhnenko and Lapa: Multi-layer Perceptron (going deeper)
  - Dreyfus: Backpropagation for training (not yet the same as SGD)
  - Amari: Use SGD for training MLPs (separating non-linearly separable patterns)
- 1970s:
  - Fukushima: Convolutional Neural Networks for images
- 1982:
  - Werbos: Modern day backpropagation / SGD

From kernels to neural networks



Two-layer neural networks

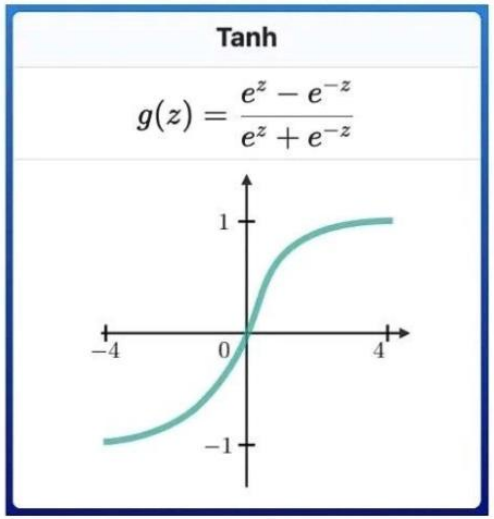
- Neural network: $S(x) = w_2^T(W_1 x + \boldsymbol{b}_1) + b_2$
  - In-class exercise: is this linear of x?
  - Still a linear model at the end of the day, so let's add a nonlinearity σ!

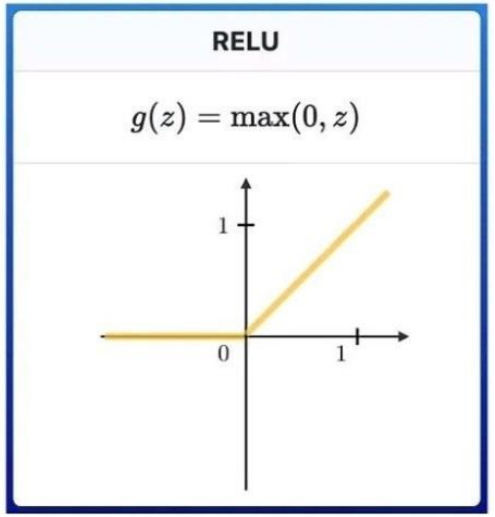
- Two-layer Multi-Layer Perceptron (MLP): $S(x) = w_2^T \sigma(W_1 x + \boldsymbol{b}_1) + b_2$
  - Suppose σ is a non-linear function
  - In-class exercise: is this linear of x?
  - Linear model w.r.t. to a learnable feature map

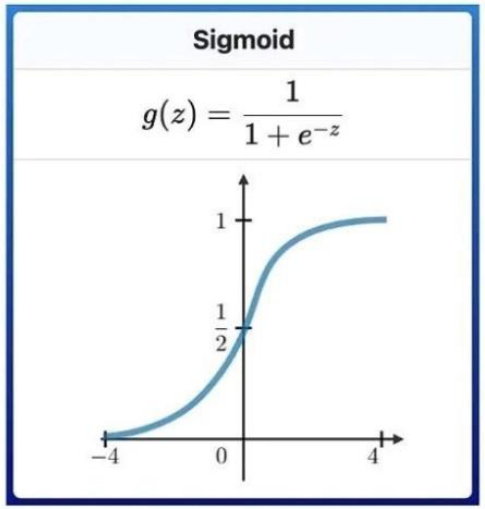
- RBF-kernel: $S(x) = w_2^T \exp\big(-\gamma(W_1 x + \boldsymbol{b}_1)\big) + b_2$

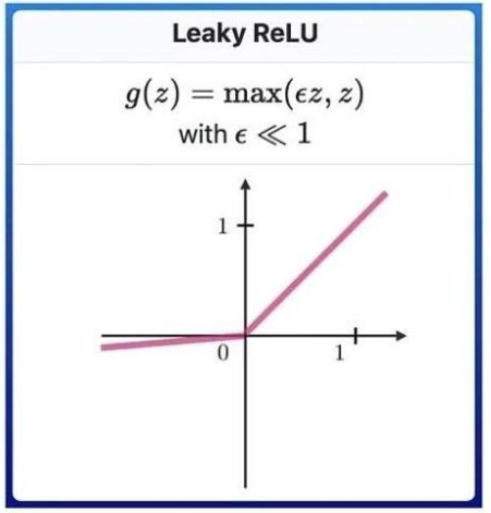
Choices of activation function

- Activation function must be non-linear!
- Building a neural network == Building a non-linear function!



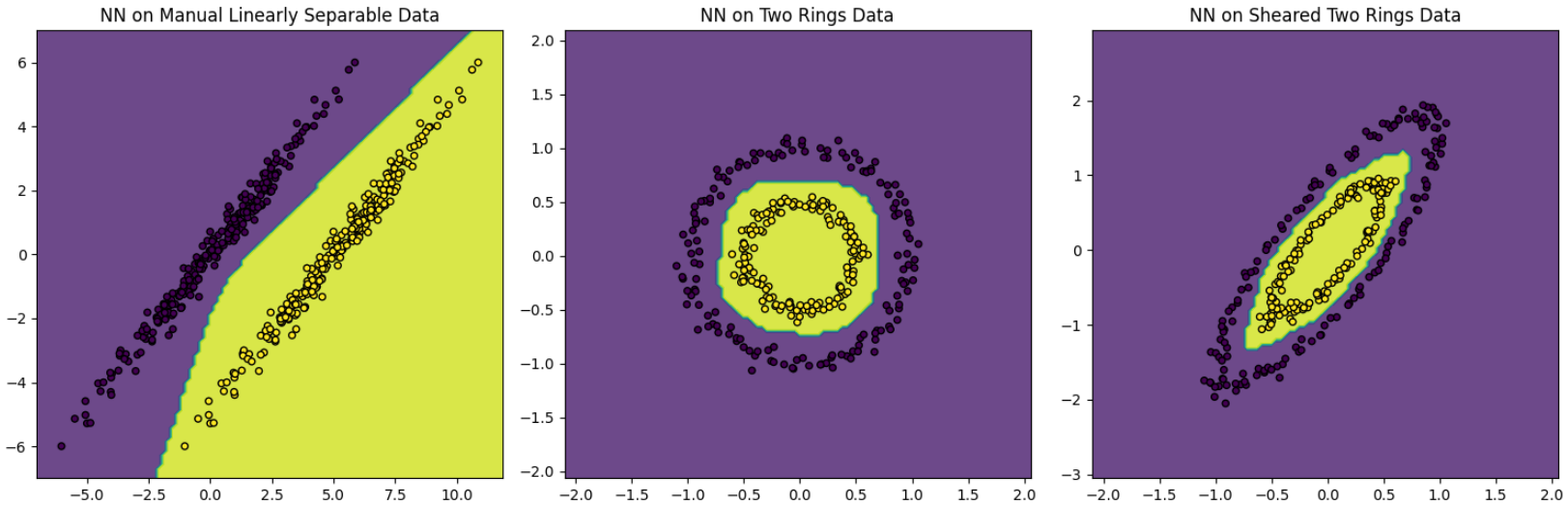






Results of fitting MLPs on our three examples

```
from sklearn.neural_network import MLPClassifier
hidden = 100
# Neural network classifier
nn_clf = MLPClassifier(hidden_layer_sizes=(hidden,), activation='relu', max_iter=1000)
```

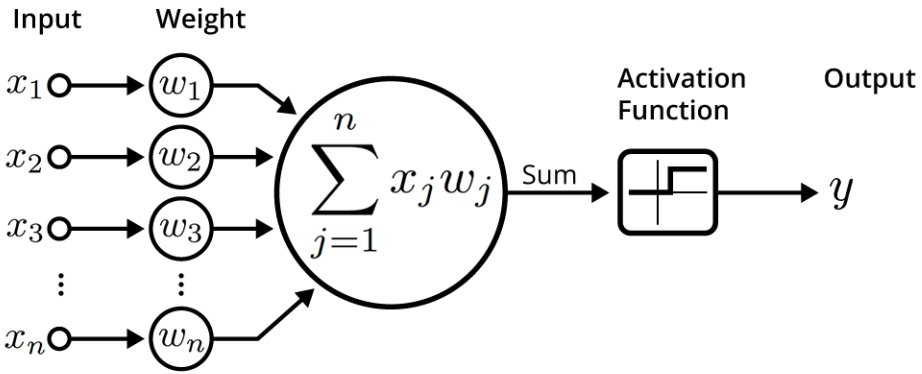


How to train a two-layer neural network?

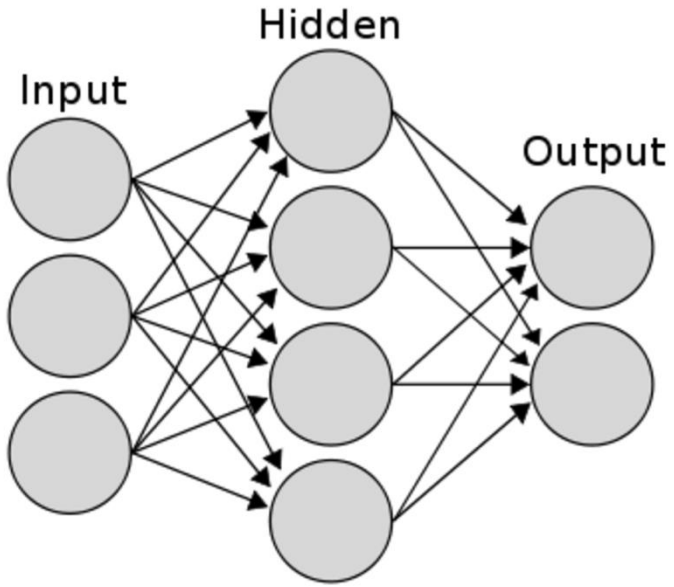
- Two-layer NN: $S(x) = \boldsymbol{w}_2^T \sigma(W_1 x + \boldsymbol{b}_1) + b_2$
- Discussion: What are the parameters?
- Discussion: Given a regression problem, what algorithm do you want to apply here?
  - GD / SGD!

How to build a stronger neural networks? More width (More neurons in each layer)

- Two-layer MLP: $S(x) = w_2^T \sigma(W_1 x + \boldsymbol{b}_1) + b_2$
- Input and output of a neuron:

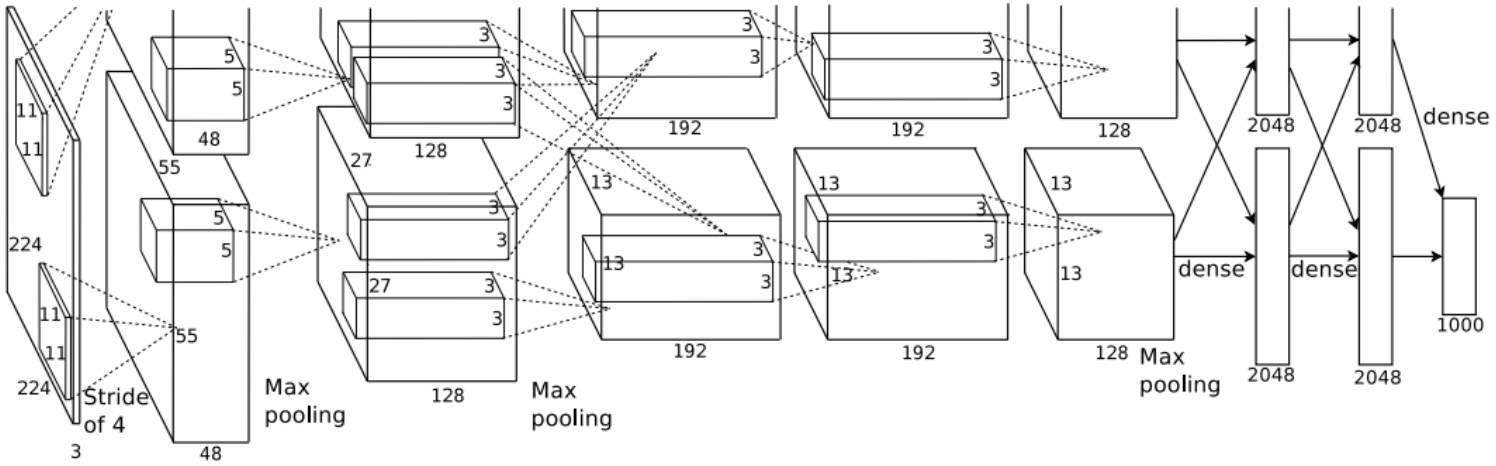


- Discussion: How many weight parameters in this fully connected NN?



How to build a stronger neural network? More depth (More layers) - Deep Learning

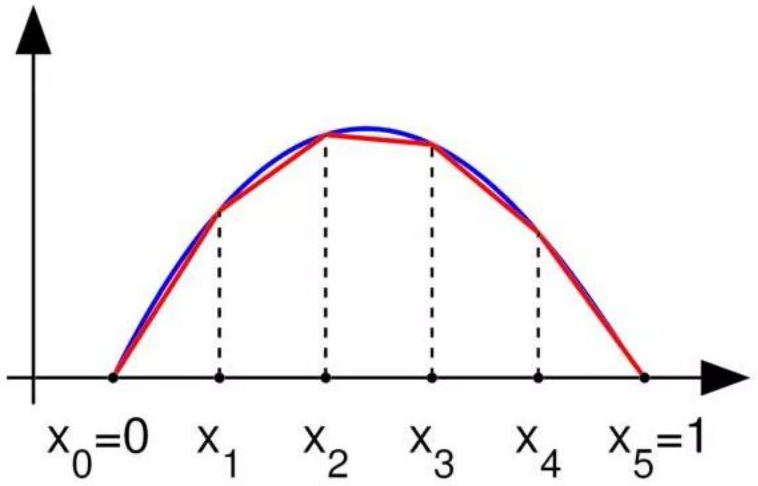
- For example: AlexNet

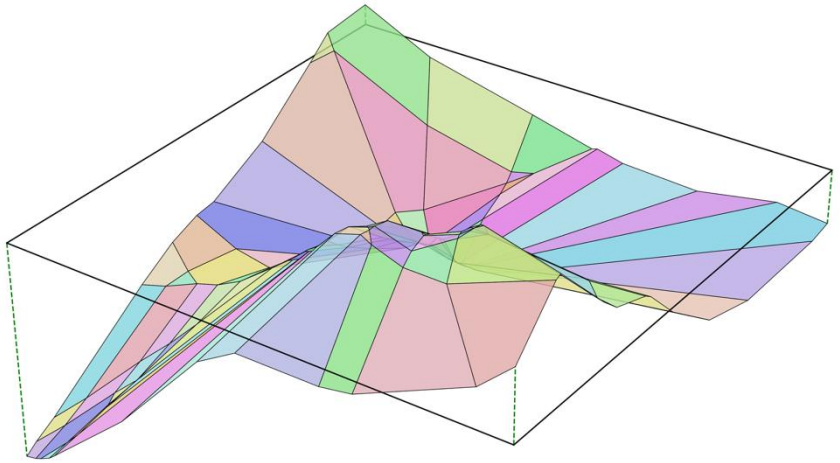


- Theoretical/visual understanding of the success of deep learning:
  - An 1-d NN $f(x)$ with width N (# of neurons in each layer), depth L (# of hidden layers), and ReLu activation function can model a piece-wise linear function of $O(N^L)$ pieces.

How to build a stronger neural network? More depth (More layers) - Deep Learning

- Theoretical understanding of the success of deep learning:
  - An 1-d NN $f(x)$ with width N (# of neurons in each layer), depth L (# of hidden layers), and ReLu activation function can model a piece-wise linear function of $O(N^L)$ pieces.
- 1-d and 2-d piece-wise linear functions:





Deep Learning

- Some useful facts about deep learning to know:

1. Non-linear activation function.
2. Feature expansion technique but with learned features.
3. You choose a deep learning model for constructing hypothesis classes that are suitable for your problem.
4. Training process requires a lot of computational resources.

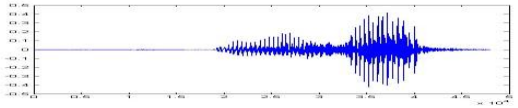
You can use deep learning for all kinds of ML problems: classification, regression, clustering, dimension reduction etc..

- Deep learning provides a learnable function approximation

- Different kinds of architecture (like LEGO blocks) are designed to address different challenges in different kind of problems:
    - Feedforward neural network
    - Recurrent neural network
    - Boltzmann machine
    - Convolutional neural network
    - Graph Neural Networks
    - Transformers
    - …

Learning ≈ Configuring the learnable function so it behaves as instructed.

- Speech Recognition

$f($  $) =$ “Hello!”

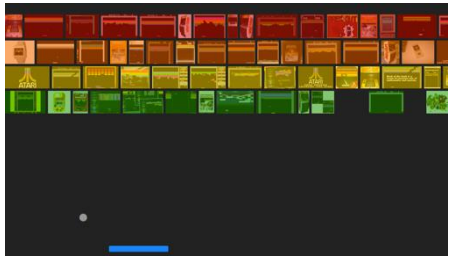
- Handwritten Recognition

$f($  $) =$ “2”

- Weather forecast

$f($ Thursday $) =$ “  Saturday”

- Play video games

$f($  $) =$ “move left”

Generally speaking, you need to make decisions about

- Which loss function to use
  - For regression, classification, clustering, dimension reduction, but also ranking, recommendation, and others…
- What type of neural network to use
  - Images
  - Text
  - Graphs (node and edges)
  - Time series
  - Decide on the hyperparameters: Depth, Width, Number of hidden units…
- How to train the neural network?
  - Initialization of weights: iid random? Rescale or not?
  - Optimizer to use: SGD, etc…
- How to collect, pre-process the data…

Modern neural networks are very complicated – ResNet (2016)

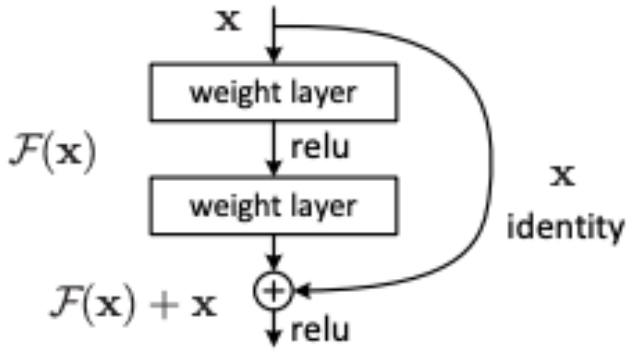
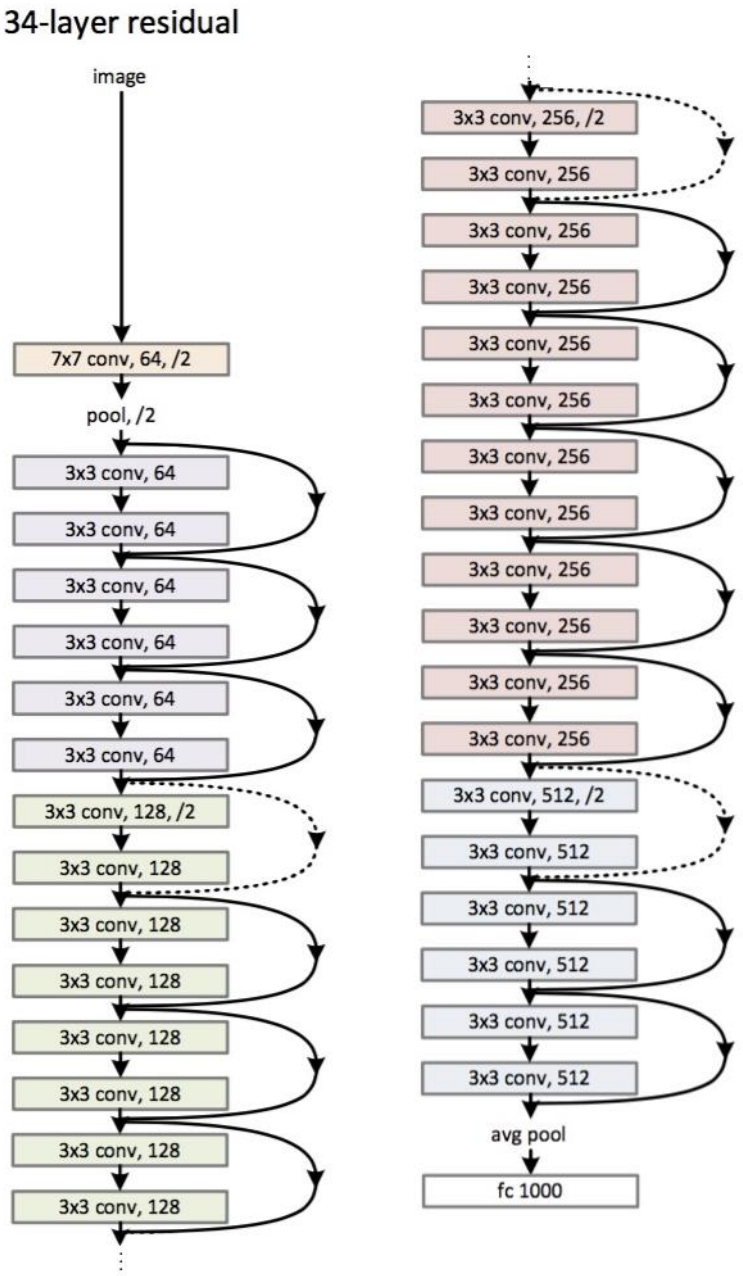
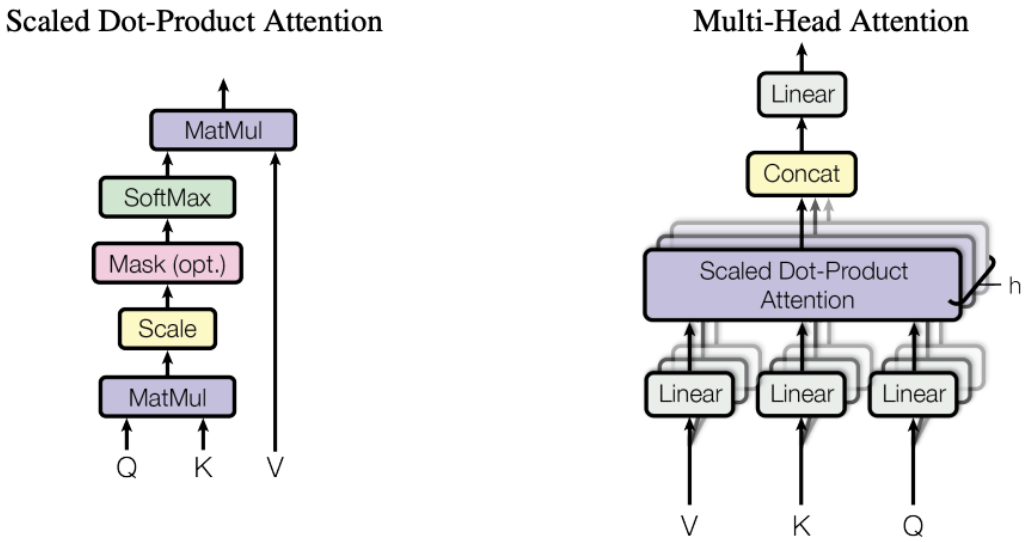


Figure 2. Residual learning: a building block.

Deep residual learning for **image recognition**
K He, X Zhang, S Ren, J Sun - … and **pattern recognition**, 2016 - openaccess.thecvf.com
… **Deeper** neural **networks** are more difficult to train. We present a **residual learning** framework to ease the training of **networks** that are substantially **deeper** than those used previously. …
☆ Save 99 Cite Cited by 189280 Related articles All 76 versions Import into BibTeX »



Modern neural networks are very complicated – Transformer (2017)



Attention is all you need

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

… to attend to **all** positions in the decoder up to and including that position. **We need** to prevent … **We** implement this inside of scaled dot-product **attention** by masking out (setting to −∞) …

☆ Save 99 Cite Cited by 97503 Related articles All 62 versions Import into BibTeX »

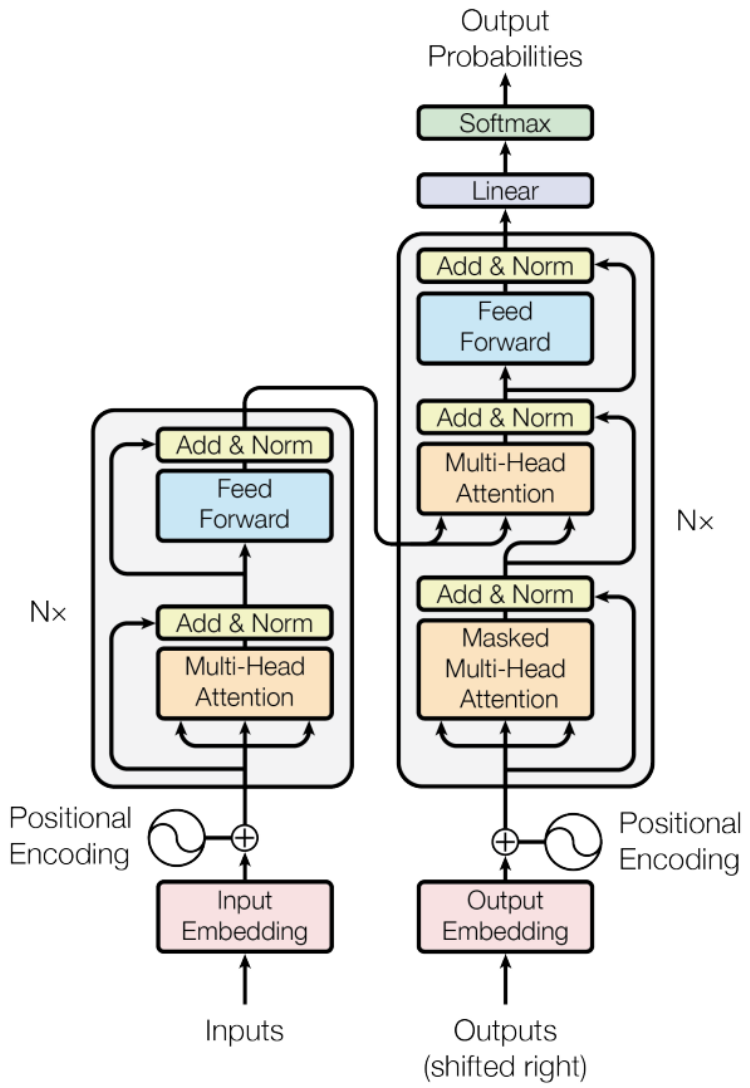


Figure 1: The Transformer - model architecture.

Solution to this? Brute-force computation with autograd and GPUs

- **Autograd**: basically chain rules, can be automated.
  - Design networks such that every block is differentiable.

- **Faster Computation**:
  - Well-packaged Deep Learning Farmework: Write Python wrapper code but running C++ underneath
  - Parallel computing: Numerical linear algebra and GPUs, scientific computing, supercomputing centers.
  - Distributed computing: Cloud computing, Map-Reduce, federated learning

- Popular tools (there are many more of these):

PyTorch

tensorflow



JAX: Autograd and XLA

Summary

- Neural network
  - Learning with neural network == fitting a neural network function
  - How to build a strong neural network with great learning ability?
    - Non-linear activation function
    - More layers
- Deep learning
  - Deep learning models are deep neural networks with many many layers
  - Its training process requires a lot of resources
  - It can be used for all kinds of ML problems
- Create non-linear hypothesis:
  - Ensemble methods (bagging, boosting)
  - Neural network
- Transform feature representation:
  - Kernel methods
  - Neural network

A brief history of machine learning

- We are now in a great time of machine learning / AI!

Year	Leading models	Built on	Hot research topics
1990s-2012	SVM	Linear models	Kernel methods Statistical machine learning
	Boosting	Decision trees	Ensemble learning Learning theory
2012	Deep neural networks	Neural networks	Deep learning Computer vision
2017	Transformer	Deep neural network	Natural language processing
2022	Generative Pre-trained Transformer (GPT)	Transformer	Large Language Model (LLM) Generative AI (GenAI)
2027	???	GPT ?	???